CRYPTOGRAPHY AND NETWORK SECURITY

# UNIT II

## Symmetric and Asymmetric Key Ciphers

**Syllabus covered**

**Symmetric key ciphers:** block cipher principles, Data Encryption Standard (DES), Advanced Encryption Standard (AES), Blowfish, RC5 and IDEA.

**Asymmetric key ciphers:** principles of public-key cryptosystems, RSA algorithm, ElGamal cryptography, Diffie–Hellman key exchange and the Knapsack algorithm.

# Contents at a Glance

## Part A – Symmetric Key Ciphers


- 1. Symmetric encryption basics and cryptanalysis
- 2. Block cipher principles (confusion, diffusion, Feistel structure)
- 3. Data Encryption Standard (DES) and Triple DES
- 4. Advanced Encryption Standard (AES)
- 5. Blowfish
- 6. RC5
- 7. IDEA
- 8. Comparison of the symmetric ciphers


## Part B – Asymmetric Key Ciphers


- 9. Principles of public-key cryptosystems (with the mathematics you need)
- 10. RSA algorithm
- 11. ElGamal cryptography
- 12. Diffie–Hellman key exchange
- 13. Knapsack algorithm (Merkle–Hellman)
- 14. Comparison, formula sheet and review questions

# PART A – SYMMETRIC KEY CIPHERS

## 1. Symmetric Encryption Basics

**Symmetric encryption** (also called conventional, secret-key or single-key encryption) uses the **same secret key** for both encryption and decryption. It was the only kind of encryption in use before public-key cryptography was invented in the 1970s, and it is still the workhorse for encrypting bulk data because it is fast.

### Five ingredients of a symmetric cipher

1. **Plaintext** – the original, readable message or data fed to the algorithm.
2. **Encryption algorithm** – performs substitutions and transformations on the plaintext.
3. **Secret key** – an input to the algorithm, independent of the plaintext. The exact substitutions and transformations depend on it.
4. **Ciphertext** – the scrambled output. It depends on both the plaintext and the key.
5. **Decryption algorithm** – essentially the encryption algorithm run in reverse, producing the plaintext from the ciphertext and the same key.

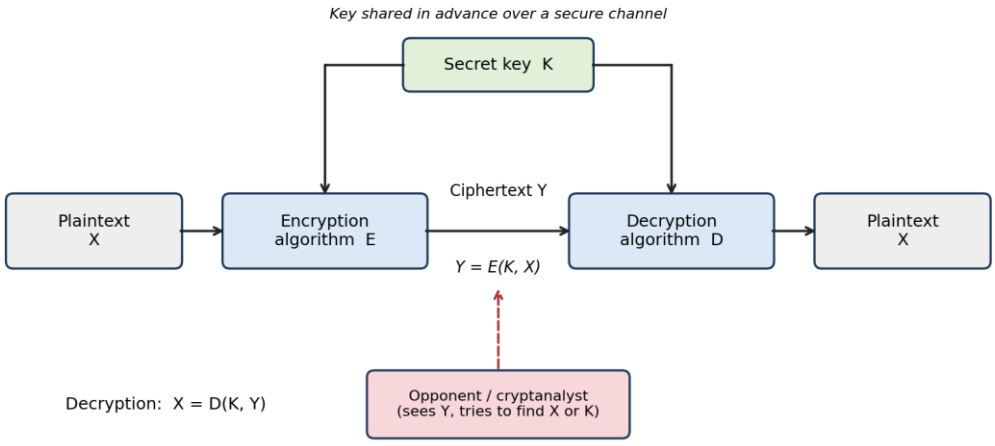



**Figure 1:** *Simplified model of symmetric encryption.*

### Two requirements for secure use

- A **strong encryption algorithm**: even if the opponent knows the algorithm and has several ciphertext–plaintext pairs, they must not be able to find the key or decrypt other messages.
- A **secure way to distribute the key**: sender and receiver must obtain copies of the secret key securely and keep it safe. The security rests on the secrecy of the key, not on the secrecy of the algorithm (Kerckhoffs' principle), so algorithms such as DES and AES are public.

### Cryptanalysis: types of attack

| Attack type | What the cryptanalyst knows |
|---|---|
| Ciphertext only | Encryption algorithm and the ciphertext to be decoded |
| Known plaintext | Algorithm, ciphertext, and one or more plaintext–ciphertext pairs made with the secret key |

| Attack type | What the cryptanalyst knows |
|---|---|
| Chosen plaintext | Algorithm, ciphertext, and a plaintext message **chosen by the analyst** with its ciphertext |
| Chosen ciphertext | Algorithm, ciphertext, and a ciphertext **chosen by the analyst** with its decrypted plaintext |
| Chosen text | Combination of chosen plaintext and chosen ciphertext |

An encryption scheme is **unconditionally secure** if the ciphertext does not contain enough information to determine the plaintext, however much computing power is available (only the one-time pad achieves this). It is **computationally secure** if the cost of breaking it exceeds the value of the information, or the time required exceeds the useful lifetime of the information. Real ciphers aim to be computationally secure. The simplest attack is **brute force**, trying every possible key:

| Key size (bits) | Number of keys | Time at 1 decryption/µs | Time at $10^6$ decryptions/µs |
|---|---|---|---|
| 32 | $2^{32} \approx 4.3 \times 10^9$ | 35.8 minutes | 2.15 milliseconds |
| 56 (DES) | $2^{56} \approx 7.2 \times 10^{16}$ | 1142 years | 10.01 hours |
| 128 (AES) | $2^{128} \approx 3.4 \times 10^{38}$ | $5.4 \times 10^{24}$ years | $5.4 \times 10^{18}$ years |

*Times are for searching half the key space on average.*

# 2. Block Cipher Principles

## 2.1 Stream ciphers versus block ciphers

- A **stream cipher** encrypts a digital data stream one bit or one byte at a time (examples: the autokeyed Vigenère cipher, Vernam cipher, RC4).
- A **block cipher** treats the plaintext as fixed-size blocks and produces a ciphertext block of equal length for each. Typical block sizes are 64 or 128 bits. DES, AES, Blowfish, RC5 and IDEA are all block ciphers; most of the symmetric ciphers in use today are block ciphers.

## 2.2 The ideal block cipher and why we cannot build it

A block cipher with an n-bit block maps each of the $2^n$ possible plaintext blocks to a distinct ciphertext block (it must be reversible, i.e. a permutation). In the **ideal block cipher** the key simply selects one of the $2^n!$ possible permutations. For a small block (n = 4) the key is the 16-entry lookup table, which is fine. For n = 64 the table would need $64 \times 2^{64} \approx 10^{21}$ bits, so the key would be astronomically large. Therefore practical ciphers **approximate** the ideal cipher using a short key to drive a structured, repeatable process.

Claude Shannon (1949) proposed doing this by combining two simple operations in a **product cipher** – a cipher that repeats substitution and permutation steps. This is the basis of the **substitution–permutation network (SPN)** used by AES and of the Feistel network used by DES.

## 2.3 Confusion and diffusion

| Property | Meaning | How it is achieved |
|---|---|---|
| **Diffusion** | The statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext. Each plaintext bit should affect many ciphertext bits. | Permutations / linear mixing, repeated over many rounds (so a one-bit change causes about half the output bits to change – the **avalanche effect**). |
| **Confusion** | The relationship between the key and the ciphertext is made as complex as possible, so the key cannot be deduced from the ciphertext. | Complex non-linear substitutions (S-boxes) that depend on the key. |

## 2.4 Feistel cipher structure

Horst Feistel (IBM) proposed in 1973 a structure that builds a strong block cipher by alternating substitutions and permutations. It is the framework behind DES, Blowfish, RC5 (in a Feistel-like form) and many other ciphers. Its big practical advantage is that the round function F *does not need to be invertible*, yet decryption still works.

### Algorithm

The plaintext block of 2w bits is split into a left half $L_0$ and a right half $R_0$, each w bits long. The two halves go through n **rounds** of processing. In round i, the right half and the subkey $K_i$ are fed to the round function F, and the output is XORed with the left half; the halves are then exchanged:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

After the last round the halves are swapped once more (so the output is $R_n \,||\, L_n$) and the ciphertext block is formed.

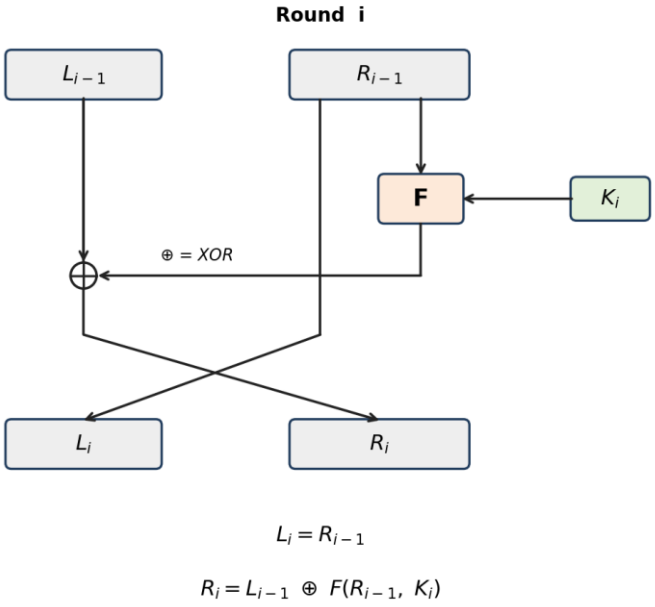



**Figure 2:** *One round of a Feistel cipher.*

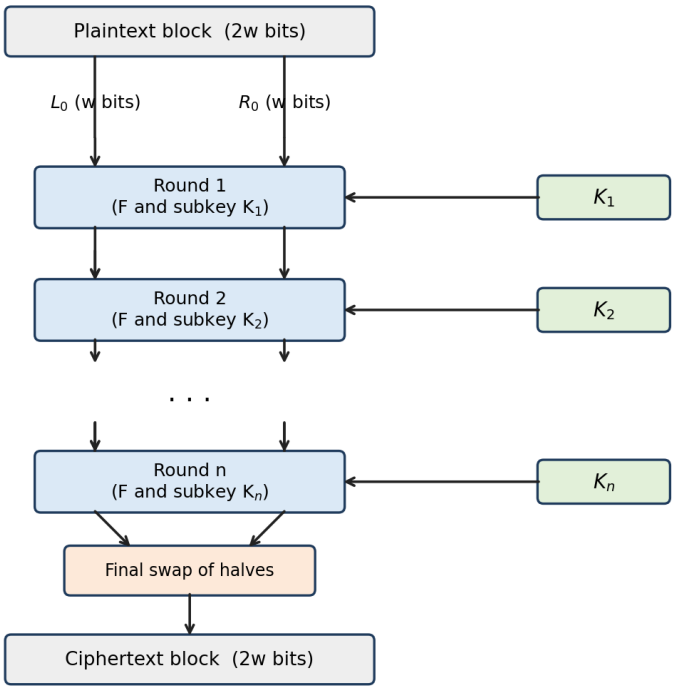



**Figure 3:** *Feistel network with n rounds (all subkeys K_1 … K_n derived from the master key).*

### Decryption is the same algorithm

To decrypt, feed the ciphertext into the **same** structure but use the subkeys in **reverse order** ($K_n$ first, $K_1$ last). Why this works, for the last encryption round (round 16 in a 16-round cipher): the ciphertext is ($RE_{16}$ || $LE_{16}$) where $LE_{16} = RE_{15}$ and $RE_{16} = LE_{15} \oplus F(RE_{15}, K_{16})$. The first decryption round gets $LD_0 = RE_{16}$, $RD_0 = LE_{16}$, so

$$\mathrm{LD}_1 = \mathrm{RD}_0 = \mathrm{LE}_{16} = \mathrm{RE}_{15}$$

$$\mathrm{RD}_1 = \mathrm{LD}_0 \oplus \mathrm{F}(\mathrm{RD}_0, \mathrm{K}_{16}) = [\mathrm{LE}_{15} \oplus \mathrm{F}(\mathrm{RE}_{15}, \mathrm{K}_{16})] \oplus \mathrm{F}(\mathrm{RE}_{15}, \mathrm{K}_{16}) = \mathrm{LE}_{15}$$

The two F terms cancel because $x \oplus x = 0$. So after one decryption round we are back at the state ($RE_{15}$, $LE_{15}$), and the same argument repeats for each round.

### Design parameters of a Feistel cipher

| Parameter | Effect and typical values |
|---|---|
| Block size | Larger blocks give more security (better diffusion) but slow the cipher. 64 bits was common; 128 bits is now standard. |
| Key size | Larger keys resist brute force but may slow the cipher. 128 bits is the current minimum for new designs. |
| Number of rounds | One round is weak, many rounds give strength. 16 rounds is typical. |
| Subkey generation algorithm | More complex key scheduling makes cryptanalysis harder. |
| Round function F | A more complex F makes the cipher harder to analyse. |
| Fast software encryption/decryption | Often the cipher is embedded in applications, so speed matters. |
| Ease of analysis | A cipher that is easy to explain is easier to analyse for weaknesses. |

# 3. Data Encryption Standard (DES)

## 3.1 Background

- DES was developed at IBM (from the earlier **Lucifer** cipher) and adopted by the US National Bureau of Standards (now NIST) as **FIPS 46** in 1977. It was the most widely used encryption scheme for over 20 years.
- It is a **Feistel cipher** with a **64-bit block** and a **56-bit key** (a 64-bit key in which every 8th bit is a parity bit), with **16 rounds**.
- The algorithm is the same for encryption and decryption; only the order of the 16 subkeys changes.

## 3.2 General structure of DES

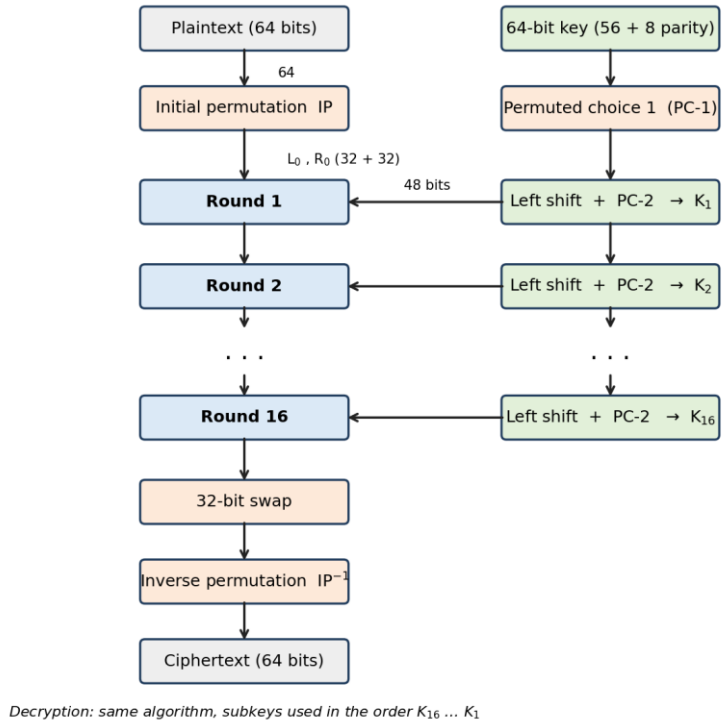



**Figure 4:** *General depiction of DES encryption (left) and key schedule (right).*

The processing of a plaintext block has three phases:

1. The 64-bit plaintext passes through an **initial permutation (IP)** that rearranges the bits. The result is split into a left half $L_0$ and a right half $R_0$, each 32 bits.
2. Sixteen **rounds** of the Feistel function are applied. Each round uses a 48-bit subkey $K_i$ derived from the 56-bit key.

3. The halves are swapped (the **32-bit swap**), and the **inverse initial permutation IP$^{-1}$** produces the 64-bit ciphertext.

### Initial permutation (IP)

The table is read left to right, top to bottom: the output bit 1 is input bit 58, output bit 2 is input bit 50, and so on. IP$^{-1}$ undoes it. IP and IP$^{-1}$ have no cryptographic value (they were meant to ease loading into 1970s hardware); security comes from the 16 rounds.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57 | 49 | 41 | 33 | 25 | 17 | 9 | 1 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 7 |

*Initial permutation (IP)*

## 3.3 Details of a single round

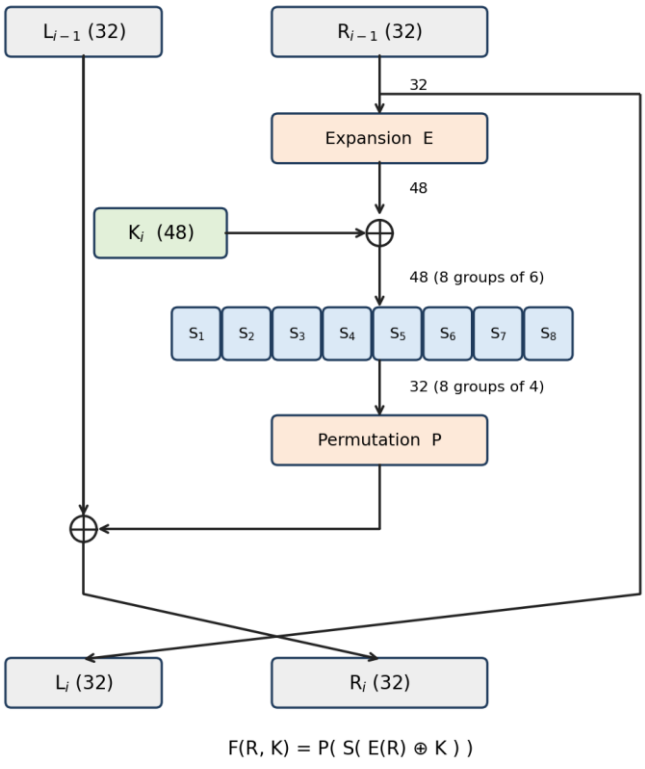



**Figure 5:** *Single round of DES (Feistel round with F = P(S(E(R) ⊕K))).*

Each round follows the Feistel equations $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$. The function F works in four steps:

1. **Expansion E (32 → 48 bits).** The 32-bit right half is expanded to 48 bits by repeating some bits (see the table below). This lets the input affect more S-box outputs in the next round (diffusion).
2. **Key mixing.** The 48-bit result is XORed with the 48-bit round subkey $K_i$.
3. **Substitution with 8 S-boxes (48 → 32 bits).** The 48 bits are divided into eight 6-bit groups, each fed to a separate S-box that outputs 4 bits. This is the only **non-linear** step and gives the cipher its confusion.
4. **Permutation P (32 → 32 bits).** The 32 S-box output bits are permuted (table below) to spread each S-box's output over several S-boxes in the next round.

### Expansion table E and permutation P

**Expansion E (32 → 48):** each entry says which input bit is placed in that output position.

| | | | | | |
|---|---|---|---|---|---|
| 32 | 1 | 2 | 3 | 4 | 5 |
| 4 | 5 | 6 | 7 | 8 | 9 |
| 8 | 9 | 10 | 11 | 12 | 13 |
| 12 | 13 | 14 | 15 | 16 | 17 |
| 16 | 17 | 18 | 19 | 20 | 21 |
| 20 | 21 | 22 | 23 | 24 | 25 |
| 24 | 25 | 26 | 27 | 28 | 29 |
| 28 | 29 | 30 | 31 | 32 | 1 |

**Permutation P (32 → 32):**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 16 | 7 | 20 | 21 | 29 | 12 | 28 | 17 |
| 1 | 15 | 23 | 26 | 5 | 18 | 31 | 10 |
| 2 | 8 | 24 | 14 | 32 | 27 | 3 | 9 |
| 19 | 13 | 30 | 6 | 22 | 11 | 4 | 25 |

### S-boxes

Each S-box is a table with **4 rows (0–3) and 16 columns (0–15)**; each entry is a 4-bit number. For a 6-bit input $b_1b_2b_3b_4b_5b_6$: the **outer bits $b_1b_6$** select the row and the **middle four bits $b_2b_3b_4b_5$** select the column. S-box $S_1$ is:

| Row \ Col | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 14 | 4 | 13 | 1 | 2 | 15 | 11 | 8 | 3 | 10 | 6 | 12 | 5 | 9 | 0 | 7 |
| 1 | 0 | 15 | 7 | 4 | 14 | 2 | 13 | 1 | 10 | 6 | 12 | 11 | 9 | 5 | 3 | 8 |
| 2 | 4 | 1 | 14 | 8 | 13 | 6 | 2 | 11 | 15 | 12 | 9 | 7 | 3 | 10 | 5 | 0 |
| 3 | 15 | 12 | 8 | 2 | 4 | 9 | 1 | 7 | 5 | 11 | 3 | 14 | 10 | 0 | 6 | 13 |

**Worked S-box lookup**

Input to $S_1$ = **011011**. Outer bits = 0 and 1 → binary 01 = row **1**. Middle bits = 1101 = column **13**.

Row 1, column 13 of $S_1$ is **5** = binary **0101**. So the 6-bit input 011011 is replaced by the 4-bit output 0101.

## 3.4 Key generation (key schedule)

1. The 64-bit key (including 8 parity bits) goes through **Permuted Choice 1 (PC-1)**, which discards the parity bits and permutes the remaining 56 bits. The result is split into two 28-bit halves $C_0$ and $D_0$.
2. In each round i, C and D are **rotated left** separately by one or two bit positions according to the schedule below, giving $C_i$ and $D_i$.
3. **Permuted Choice 2 (PC-2)** selects 48 of the 56 bits of $C_i \,||\, D_i$ to form the subkey $K_i$.

| Round | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Left shifts | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 |

The shifts total 28 positions, so after round 16 the registers are back to $C_0$, $D_0$. Because the rotation schedule is fixed, decryption can produce the subkeys in reverse order ($K_{16}$ first) by rotating right instead.

## 3.5 DES decryption

Use the same hardware or code: input the ciphertext, apply IP, run the 16 rounds using the subkeys in the order $K_{16}, K_{15}, \ldots, K_1$, swap halves, then apply $IP^{-1}$. The result is the original plaintext. (This follows directly from the Feistel proof in section 2.4.)

## 3.6 The avalanche effect

A desirable property of any block cipher is that a small change in plaintext or key produces a large change in ciphertext. In DES, changing a single plaintext bit (or key bit) changes about half of the 64 ciphertext bits after only a few rounds. This is the combined result of expansion E, the S-boxes and permutation P.

## 3.7 Strength of DES

- **Use of 56-bit keys.** The key space is $2^{56} \approx 7.2 \times 10^{16}$. In 1998 the Electronic Frontier Foundation's special-purpose "DES cracker" recovered a key in less than three days, and a combined effort in 1999 took about 22 hours. DES is therefore considered **insecure by brute force**.
- **Nature of the algorithm.** Critics worried about hidden weaknesses in the S-boxes. Later work showed that the S-boxes were designed to resist **differential cryptanalysis**, which was known to the designers but not public.
- **Differential cryptanalysis** (Biham and Shamir) needs about $2^{47}$ chosen plaintexts to break 16-round DES; **linear cryptanalysis** (Matsui) needs about $2^{43}$ known plaintexts. Both are

impractical compared with simply buying key-search hardware, so the key length remains the main weakness.

- **Timing attacks** exploit the time taken to run the algorithm; they are more of a threat to some implementations (for example smart cards) than to the algorithm itself.

# 4. Advanced Encryption Standard (AES)

## 4.1 Background

- By the 1990s DES's 56-bit key and 3DES's slowness and 64-bit block called for a replacement. In 1997 NIST issued a public call for the **Advanced Encryption Standard**; the winner, **Rijndael** by the Belgian cryptographers **Joan Daemen and Vincent Rijmen**, was selected in 2000 and published as **FIPS 197** in 2001.
- NIST's requirements: a symmetric block cipher with a **128-bit block** and key lengths of **128, 192 and 256 bits**, unclassified, royalty-free worldwide, and more efficient and more secure than 3DES.
- AES is **not a Feistel cipher**. It is a **substitution–permutation network** that processes the **whole block in parallel** in every round, using byte-oriented operations over the finite field GF($2^8$).

| Parameter | AES-128 | AES-192 | AES-256 |
|---|---|---|---|
| Key size (words / bytes / bits) | 4 / 16 / 128 | 6 / 24 / 192 | 8 / 32 / 256 |
| Plaintext block size (words / bits) | 4 / 128 | 4 / 128 | 4 / 128 |
| Number of rounds | 10 | 12 | 14 |
| Round key size (words / bits) | 4 / 128 | 4 / 128 | 4 / 128 |
| Expanded key size (words) | 44 | 52 | 60 |

*1 word = 32 bits = 4 bytes.*

## 4.2 Overall structure

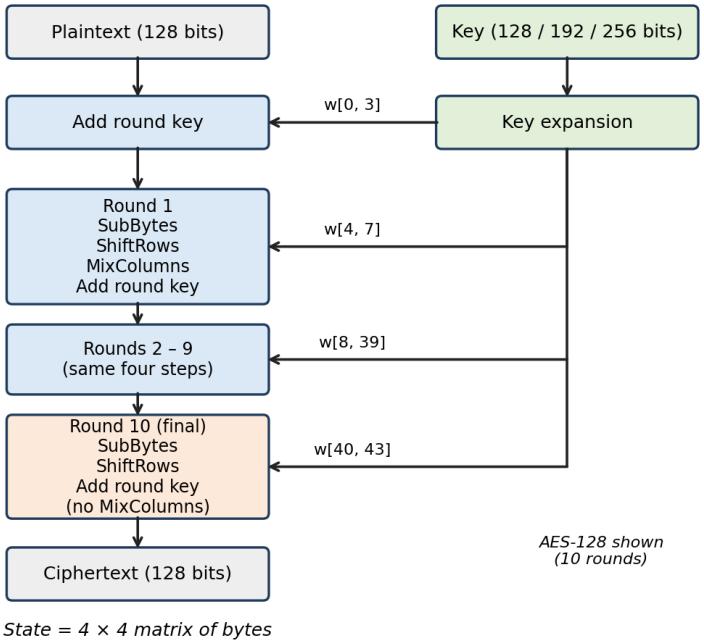



**Figure 7:** *AES encryption process (128-bit key shown).*

The 128-bit input block is copied into a 4 × 4 matrix of bytes called the **State**, filled **column by column** (the first four input bytes form the first column). The State is modified by each stage and is copied to the output at the end. The cipher consists of:

- An initial **AddRoundKey** using the first four words of the expanded key.
- $N_r - 1$ **full rounds** (9, 11 or 13) each made of SubBytes, ShiftRows, MixColumns and AddRoundKey.
- A **final round** (round 10, 12 or 14) with SubBytes, ShiftRows and AddRoundKey, but **no MixColumns**. Leaving MixColumns out of the last round makes the decryption structure symmetrical and does not weaken the cipher, as MixColumns after the last key addition could simply be undone by an attacker.

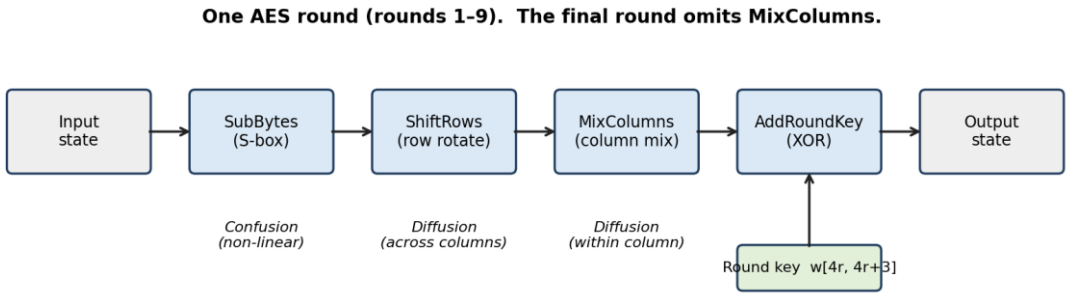



**Figure 8:** *The four transformations in one AES round.*

Only AddRoundKey uses the key; the other three are keyless but they provide the confusion and diffusion. That is why the cipher both begins and ends with AddRoundKey – any other stage at the start or end could be undone without knowing the key.

## 4.3 The four transformations

### (a) SubBytes – byte substitution

Each byte of the State is replaced independently by another byte using a fixed 16 × 16 lookup table, the **S-box**. The byte's upper hex digit selects the row and the lower digit selects the column. Examples: S(00) = **63**, S(01) = **7C**, S(53) = **ED**.

The S-box is built by (1) taking the **multiplicative inverse in GF($2^8$)** (with 00 mapped to 00), which is highly non-linear, and then (2) applying an **affine transformation** over GF(2) that removes simple algebraic structure. Properties: it is invertible (so decryption uses an Inverse S-box), has no fixed point (S(a) ≠ a) and no "opposite" fixed point (S(a) ≠ complement of a). SubBytes provides **confusion**.

### (b) ShiftRows – row permutation

Row 0 of the State is unchanged. Row 1 is rotated **1 byte to the left**, row 2 by **2 bytes** and row 3 by **3 bytes**. The inverse operation rotates to the right.

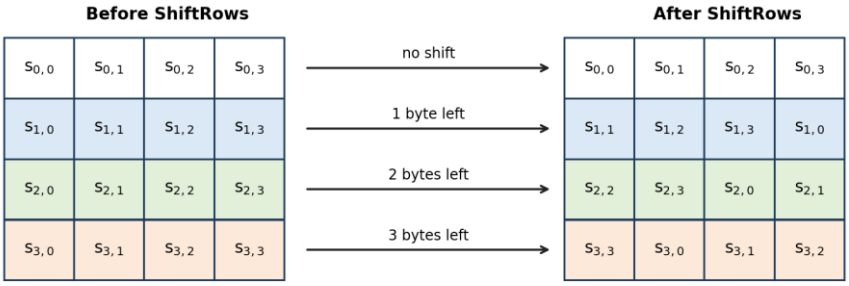



**Figure 9:** *ShiftRows: row r is rotated left by r bytes.*

ShiftRows ensures that the four bytes of one input column are spread across four different columns of the output; together with MixColumns this gives **diffusion** across the whole block after a couple of rounds.

### (c) MixColumns – column mixing

Each column of the State is treated as a four-term vector and multiplied by a fixed 4 × 4 matrix, with arithmetic performed in GF($2^8$). With input column ($s_0$, $s_1$, $s_2$, $s_3$):

| | | | | | |
|---|---|---|---|---|---|
| 02 | 03 | 01 | 01 | × $s_{0,c}$ | = $s'_{0,c}$ |
| 01 | 02 | 03 | 01 | × $s_{1,c}$ | = $s'_{1,c}$ |
| 01 | 01 | 02 | 03 | × $s_{2,c}$ | = $s'_{2,c}$ |
| 03 | 01 | 01 | 02 | × $s_{3,c}$ | = $s'_{3,c}$ |

**Arithmetic in GF($2^8$).** Bytes are polynomials over GF(2); addition is XOR; multiplication is polynomial multiplication reduced modulo the irreducible polynomial **m(x) = $x^8 + x^4 + x^3 + x + 1$** (hex 11B). Multiplying by 01 changes nothing; multiplying by **02** (called **xtime**) is a one-bit left shift, followed by XOR with **1B** if the bit shifted out was 1; and 03·a = (02·a) ⊕a.

The inverse (InvMixColumns) uses the matrix with entries 0E, 0B, 0D, 09. MixColumns provides **diffusion**: each output byte depends on all four input bytes of the column.

### (d) AddRoundKey

The 128 bits of the State are XORed with the 128-bit round key (four 32-bit words, one per column). It is its own inverse and is the only step that brings in the secret key.

## 4.4 AES key expansion

The cipher key is expanded into an array w of 4·($N_r$+1) words (44 words for AES-128). The first $N_k$ words are the key itself. Each later word depends on the previous word and the word $N_k$ positions earlier:

- **RotWord** rotates the four bytes of a word one byte to the left: [b0, b1, b2, b3] → [b1, b2, b3, b0].
- **SubWord** applies the AES S-box to each of the four bytes.
- **Rcon[j]** = (RC[j], 00, 00, 00) with RC[j] = $x^{j-1}$ in GF($2^8$): 01, 02, 04, 08, 10, 20, 40, 80, 1B, 36. It removes symmetry between the rounds.

## 4.5 AES decryption

AES decryption is **not** the same circuit as encryption (it is not Feistel). It applies the inverse transformations in reverse order with the round keys reversed:

$$\text{AddRoundKey} \rightarrow [\text{InvShiftRows} \rightarrow \text{InvSubBytes} \rightarrow \text{AddRoundKey} \rightarrow \text{InvMixColumns}] \times (N_r - 1) \rightarrow \text{InvShiftRows} \rightarrow \text{InvSubBytes} \rightarrow \text{AddRoundKey}$$

InvShiftRows and InvSubBytes commute, and by changing the key schedule (applying InvMixColumns to the middle round keys) one obtains an **equivalent inverse cipher** whose round structure matches encryption – convenient for hardware.

## 4.6 Strength and implementation notes

- No practical attack on full AES is known; the best published attacks (for example biclique attacks) reduce the work only marginally below brute force (about $2^{126}$ for AES-128), which is still computationally infeasible.
- Side-channel attacks (timing, cache, power analysis) target implementations and are countered with constant-time code or hardware instructions such as Intel/AMD **AES-NI**.
- AES is efficient in 8-bit microcontrollers (small tables) and in 32-bit software (table lookups combining SubBytes, ShiftRows and MixColumns).
- AES is the default choice for new systems (TLS, IPsec, disk encryption, Wi-Fi WPA2/WPA3). It must be combined with a proper **mode of operation** (for example CBC, CTR or GCM) when messages are longer than one block.

# 5. Blowfish

## 5.1 Overview

Blowfish was designed by **Bruce Schneier** in 1993 as a fast, free replacement for DES and IDEA. It is a **16-round Feistel cipher** on **64-bit blocks** with a **variable key length from 32 to 448 bits**. It uses **key-dependent S-boxes**, simple operations (XOR, addition mod $2^{32}$, table lookup) and is unpatented and royalty-free.

### Design features

- **Fast** – about 18 clock cycles per byte on a 32-bit processor in the original design.
- **Compact** – runs in less than 5 KB of memory.
- **Simple** – easy to code and analyse.
- **Variably secure** – the key length can be chosen to suit the application (up to 448 bits).

## 5.2 Data structures

- **P-array**: eighteen 32-bit subkeys $P_1$ … $P_{18}$.
- **Four S-boxes** $S_1$ … $S_4$, each with 256 entries of 32 bits (4 KB total).
- The P-array and S-boxes are initialised with the **fractional digits of π** (for example $P_1$ = 243F6A88, $P_2$ = 85A308D3, …) – "nothing-up-my-sleeve" numbers – and are then modified by the key.

## 5.3 Encryption

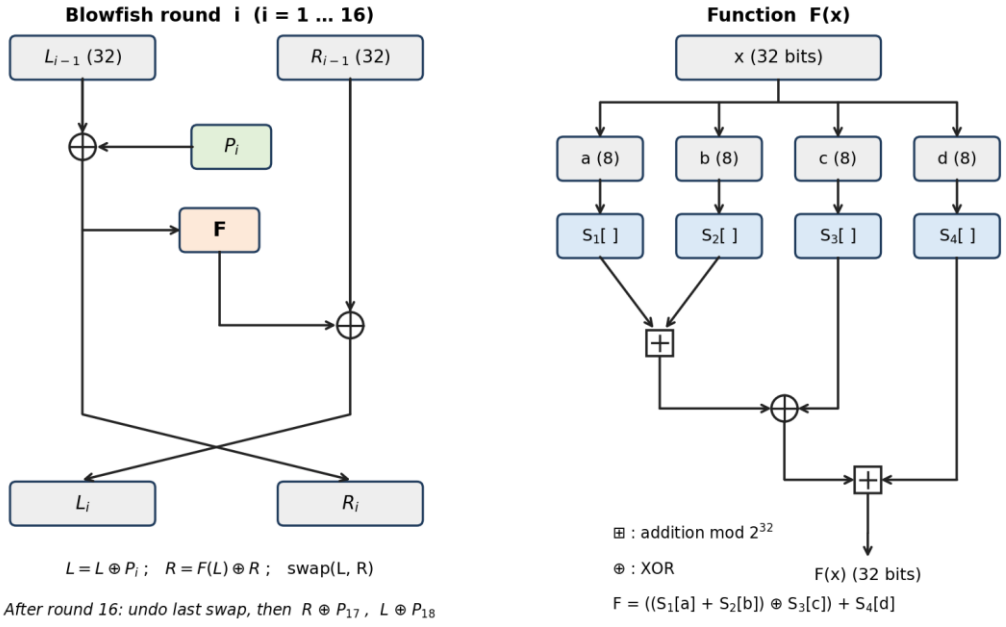



**Figure 10:** *Blowfish round (left) and the round function F (right).*

The round function splits the 32-bit input x into four bytes a, b, c, d and computes:

Mixing addition and XOR (operations from different algebraic groups) plus the large key-dependent S-boxes gives strong non-linearity. **Decryption** is identical to encryption except that $P_1$ … $P_{18}$ are used in reverse order.

## 5.4 Key expansion

1. Initialise the P-array and then the four S-boxes with the fixed digits of π.
2. XOR $P_1$ with the first 32 bits of the key, $P_2$ with the next 32 bits, and so on, cycling through the key bits until all 18 P entries have been XORed.
3. Encrypt the 64-bit all-zero block with the current P-array and S-boxes. Replace $P_1$, $P_2$ with the output.
4. Encrypt the output of step 3 with the updated tables and replace $P_3$, $P_4$.
5. Continue replacing all P entries and then, in order, all four S-box entries.

This needs $1 + 9 + 4 \times 128 =$ **521 executions** of the Blowfish algorithm (18/2 = 9 for the P-array and 1024/2 = 512 for the S-boxes), which makes key setup slow (about 4 KB of data to encrypt). That is a **feature**: it slows down brute-force key searches, but a poor fit for applications that change keys often.

# 6. RC5

## 6.1 Overview

RC5 was designed by **Ron Rivest** in 1994 (RSA Laboratories). It is a **parameterised** symmetric block cipher with a variable word size, number of rounds and key length, written **RC5-w/r/b**:

| Parameter | Meaning | Allowed values |
|---|---|---|
| w | Word size in bits. The plaintext/ciphertext block is **2w** bits. | 16, 32, 64 (block = 32, 64, 128 bits) |
| r | Number of rounds | 0 – 255 |
| b | Key length in bytes | 0 – 255 |

A common choice is **RC5-32/12/16** (32-bit words, 64-bit block, 12 rounds, 128-bit key). The design goals were: suitability for hardware and software, speed, adaptability to different word lengths, a variable number of rounds, a variable-length key, simplicity, low memory use, high security and **data-dependent rotations**, which was the novel feature intended to frustrate differential and linear cryptanalysis.

RC5 uses only three primitive operations: **addition modulo $2^w$**, **bitwise XOR**, and **left/right circular rotation by a data-dependent amount**.

## 6.2 Key expansion

The b-byte key is expanded into an array S of t = 2r + 2 words (round keys). With u = w/8 bytes per word and c = ⌈ b/u ⌉ words in the key array L:

1. **Convert** the key bytes K[0 … b−1] into the word array L[0 … c−1] (little-endian), padding with zero bytes if needed.
2. **Initialise** S using magic constants: S[0] = $P_w$; S[i] = S[i−1] + $Q_w$ mod $2^w$ for i = 1 … t−1.
3. **Mix** the key into S:

```
i = j = 0;  A = B = 0
repeat 3 * max(t, c) times:
    A = S[i] = (S[i] + A + B) <<< 3
    B = L[j] = (L[j] + A + B) <<< (A + B)
    i = (i + 1) mod t
    j = (j + 1) mod c
```

The constants come from e and the golden ratio ϕ: $P_w$ = Odd((e − 2)·$2^w$) and $Q_w$ = Odd((ϕ − 1)·$2^w$). For w = 32: **$P_{32}$ = B7E15163** and **$Q_{32}$ = 9E3779B9**. (For w = 16: B7E1 and 9E37.)

## 6.3 Encryption and decryption

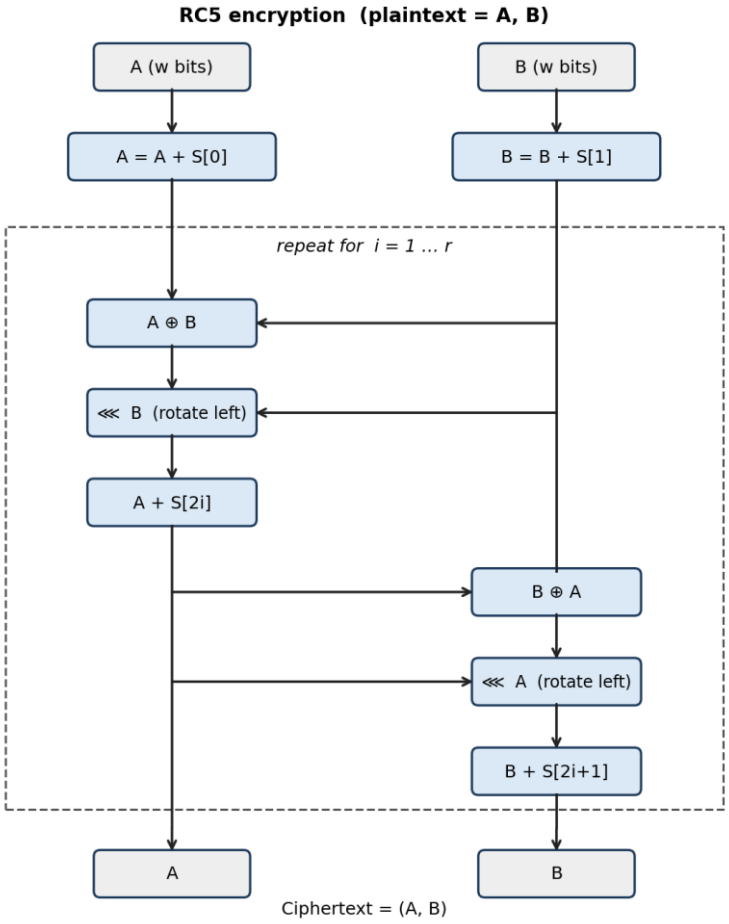



**Figure 11:** *RC5 encryption: one round (repeated r times).*

Split the plaintext into two w-bit registers A and B and compute:

```
A = A + S[0]
B = B + S[1]
for i = 1 to r:
    A = ((A XOR B) <<< B) + S[2i]
    B = ((B XOR A) <<< A) + S[2i+1]
ciphertext = (A, B)
```

Decryption runs the steps in reverse using right rotations (>>>) and subtraction:

```
for i = r down to 1:
    B = ((B - S[2i+1]) >>> A) XOR A
    A = ((A - S[2i]) >>> B) XOR B
B = B - S[1]
A = A - S[0]
```

Notice that the rotate amount is **taken from the other register** (only the low-order $\log_2 w$ bits are used), so the rotation depends on the data. A well-defined set of modes of operation is specified for RC5: ECB, CBC, CBC with padding, and ciphertext stealing (CTS).

## 6.4 Security

- The security grows with the number of rounds: 12 rounds with 32-bit words resists differential and linear attacks for practical data volumes; later recommendations use 18–20 rounds for a wider safety margin.
- Brute force on short keys is practical – a 64-bit RC5 key was recovered in 2002 by a large distributed-computing effort (distributed.net) – so use 128-bit keys or more.
- Rivest's later design **RC6** (an AES finalist) refines RC5 for 128-bit blocks.

# 7. International Data Encryption Algorithm (IDEA)

## 7.1 Overview

IDEA was designed by **Xuejia Lai and James Massey** at ETH Zurich (1991, final version 1992). It is a **64-bit block cipher with a 128-bit key** that uses **8 identical rounds plus a final output transformation**. It is **not a Feistel network**; it uses the **Lai–Massey structure**. IDEA was used in PGP (Pretty Good Privacy) before AES became common. Its main patents have expired, so it can be implemented freely.

The central design idea is to mix **three incompatible algebraic operations** on 16-bit sub-blocks, none of which is distributive or associative with respect to the others. This gives confusion without S-boxes:

| Operation | Symbol | Definition |
|---|---|---|
| Bitwise XOR | $\oplus$ | XOR of two 16-bit values |
| Addition modulo $2^{16}$ | $\boxplus$ | (a + b) mod 65536 |
| Multiplication modulo $2^{16} + 1$ | $\odot$ | (a × b) mod 65537, where the all-zero block 0000 is treated as $2^{16}$. Because 65537 is **prime**, every non-zero value has a multiplicative inverse. |

## 7.2 One round

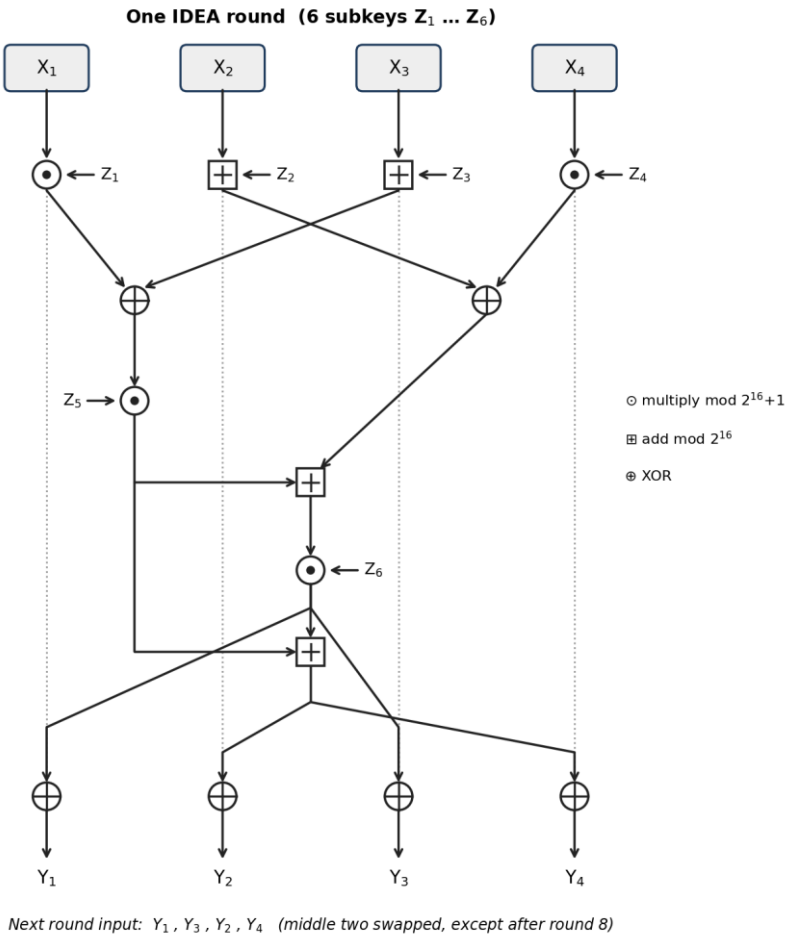



**Figure 12:** *One round of IDEA with the MA (multiplication–addition) structure.*

A round takes four 16-bit sub-blocks $X_1$, $X_2$, $X_3$, $X_4$ and six 16-bit subkeys $Z_1$ … $Z_6$. The 14 steps are:

| Step | Operation | Step | Operation |
|---|---|---|---|
| 1 | $X_1 \odot Z_1$ | 8 | step 6 $\boxplus$ step 7 |
| 2 | $X_2 \boxplus Z_2$ | 9 | step 8 $\odot Z_6$ |
| 3 | $X_3 \boxplus Z_3$ | 10 | step 7 $\boxplus$ step 9 |
| 4 | $X_4 \odot Z_4$ | 11 | step 1 $\oplus$ step 9 → new $X_1$ |
| 5 | step 1 $\oplus$ step 3 | 12 | step 3 $\oplus$ step 9 |
| 6 | step 2 $\oplus$ step 4 | 13 | step 2 $\oplus$ step 10 |
| 7 | step 5 $\odot Z_5$ | 14 | step 4 $\oplus$ step 10 → new $X_4$ |

The outputs of the round are (step 11, step 13, step 12, step 14); i.e. the **two middle values are swapped** before the next round (except after the last round). Steps 5–10 form the **multiplication–addition (MA) structure**, which is the main source of diffusion: every output depends on every input after a single round.

## 7.3 Output transformation and key schedule

- **Output transformation** (after round 8, with the last swap undone): $Y_1 = X_1 \odot Z_{49}$, $Y_2 = X_2 \boxplus Z_{50}$, $Y_3 = X_3 \boxplus Z_{51}$, $Y_4 = X_4 \odot Z_{52}$. The ciphertext is $Y_1 \,||\, Y_2 \,||\, Y_3 \,||\, Y_4$.
- **52 subkeys** are needed: 6 × 8 = 48 for the rounds plus 4 for the output transformation.
- **Key schedule:** the 128-bit key is split into eight 16-bit subkeys ($Z_1$ … $Z_8$). The 128-bit key is then **rotated left by 25 bits**, and the result again split into eight subkeys. This repeats until all 52 subkeys are produced.

## 7.4 Decryption

Decryption uses the same structure with a different set of subkeys: each decryption subkey is the **multiplicative inverse mod ($2^{16}$+1)** (for the $\odot$ keys) or the **additive inverse mod $2^{16}$** (for the $\boxplus$ keys) of an encryption subkey, taken in reverse order (the two additive keys of the middle swap are exchanged as needed). Computing a multiplicative inverse uses the extended Euclidean algorithm.

## 7.5 Security

- The 128-bit key makes brute force infeasible ($2^{128}$ keys). No practical attack against the full 8.5 rounds is known; some classes of weak keys (where many key bits are zero) exist, and a few reduced-round attacks have been published.
- IDEA is efficient in software because 16-bit operations suit many processors, but multiplication mod $2^{16}$+1 is slower than the table lookups of AES. Its 64-bit block has the same birthday-bound limitation as DES and Blowfish.

# 8. Comparison of the Symmetric Ciphers

| Cipher | Block size (bits) | Key size (bits) | Rounds | Structure | Notable features |
|---|---|---|---|---|---|
| DES | 64 | 56 | 16 | Feistel | Fixed S-boxes, IP/FP, now insecure |
| AES | 128 | 128 / 192 / 256 | 10 / 12 / 14 | SPN (not Feistel) | GF($2^8$) S-box; current standard |
| Blowfish | 64 | 32 – 448 | 16 | Feistel | Key-dependent S-boxes; slow key setup |
| RC5 | 32 / 64 / 128 | 0 – 2040 | 0 – 255 | Feistel-like | Data-dependent rotations; parameterised |
| IDEA | 64 | 128 | 8 + output | Lai–Massey | XOR, add mod $2^{16}$, multiply mod $2^{16}+1$ |

# PART B – ASYMMETRIC KEY CIPHERS

## 9. Principles of Public-Key Cryptosystems

### 9.1 Why public-key cryptography?

Symmetric encryption has two difficulties: (1) **key distribution** – two parties must somehow share a secret key before they can communicate securely, and a network of n users needs n(n−1)/2 pairwise keys; and (2) **digital signatures** – a shared key cannot prove which of the two parties produced a message. In 1976 **Whitfield Diffie and Martin Hellman** proposed a radically different approach: use **two related keys**, a public key and a private key. The first practical algorithm of this type, **RSA**, followed in 1977.

Public-key (asymmetric) algorithms rely on **mathematical functions** (number theory) rather than on substitution and permutation. They are not stronger than symmetric ciphers in any inherent sense – security depends on key length and the difficulty of the underlying mathematical problem – and they are far slower, so in practice they are used to exchange keys and sign data while symmetric ciphers encrypt the bulk data.

### 9.2 Components of a public-key cryptosystem

- **Plaintext** – the readable message input to the algorithm.
- **Encryption algorithm** – performs transformations on the plaintext.
- **Public and private keys** – a pair of keys, one used for encryption and the other for decryption. The transformations depend on which key is supplied.
- **Ciphertext** – the scrambled output; it depends on the plaintext and the key used.
- **Decryption algorithm** – accepts the ciphertext and the matching key to recover the plaintext.

#### Essential steps

1. Each user generates a **pair of keys** to be used for encrypting and decrypting messages.
2. Each user places one of the two keys in a public register or other accessible file: this is the **public key**. The companion key is kept **private**.
3. If Bob wants to send a confidential message to Alice, Bob encrypts it using Alice's **public** key.
4. When Alice receives the message, she decrypts it using her **private** key. Nobody else can decrypt it because only Alice knows her private key.

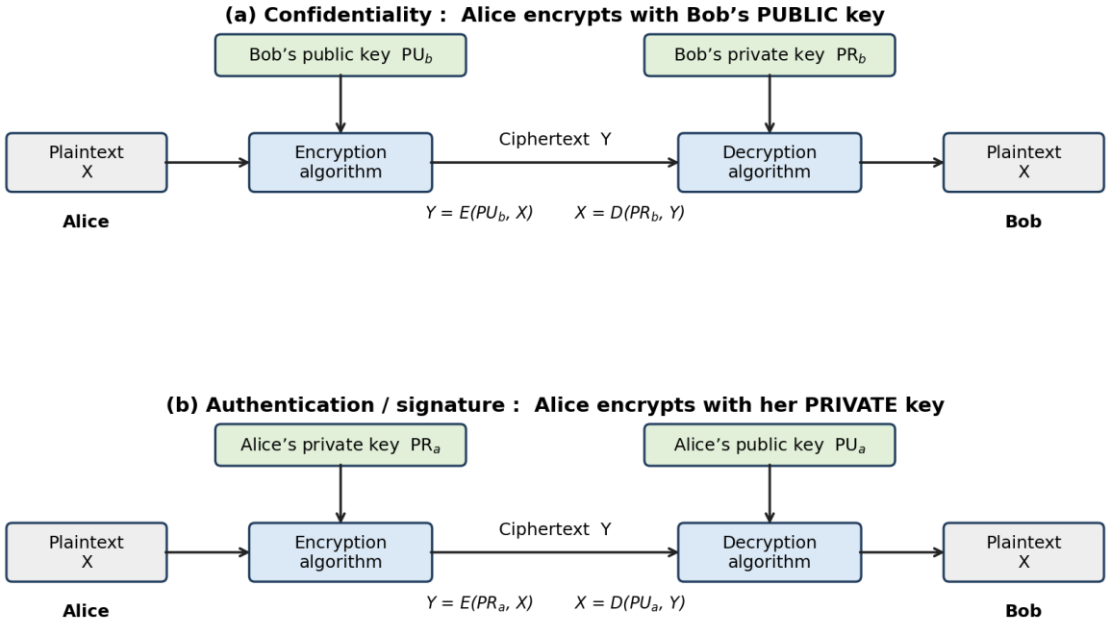



**Figure 13:** *Public-key cryptosystem: (a) confidentiality, (b) authentication.*

**Notation.** Bob's public key is $PU_b$; his private key is $PR_b$.

- **Confidentiality:** $Y = E(PU_b, X)$ and $X = D(PR_b, Y)$.
- **Authentication / digital signature:** Alice encrypts with her private key, $Y = E(PR_a, X)$; anyone can decrypt with her public key, $X = D(PU_a, Y)$. Only Alice could have produced Y, so the message is authentic and the sender cannot easily deny it (non-repudiation). In practice the signature is applied to a **hash** of the message, not the whole message.
- **Both:** $Z = E(PU_b, E(PR_a, X))$ and $X = D(PU_a, D(PR_b, Z))$.

| Conventional (symmetric) encryption | Public-key (asymmetric) encryption |
|---|---|
| **Needed to work:** the same algorithm and key for encryption and decryption; sender and receiver share the algorithm and the key. | **Needed to work:** one algorithm for encryption and decryption with a pair of keys, one for each; sender and receiver each have one of the matched pair. |
| **Needed for security:** the key must be kept secret; it should be impractical to decipher a message without further information. | **Needed for security:** one of the two keys must be kept secret; it must be impossible or impractical to decipher the message from the public key and ciphertext. |
| Fast; suited to bulk data. | Slow (heavy arithmetic); suited to small data such as keys and hashes. |
| Key distribution is a problem. | No secret needs to be sent, but public keys must be authenticated (certificates). |

## 9.3 Applications of public-key cryptosystems

- **Encryption/decryption** – the sender encrypts a message with the recipient's public key.
- **Digital signature** – the sender signs a message (or its hash) with the sender's private key.

- **Key exchange** – two parties cooperate to establish a shared session key.

| Algorithm | Encryption / decryption | Digital signature | Key exchange |
|---|---|---|---|
| RSA | Yes | Yes | Yes |
| ElGamal | Yes | Yes (ElGamal signature) | Yes (via Diffie–Hellman) |
| Diffie–Hellman | No | No | Yes |
| DSS | No | Yes | No |
| Elliptic curve | Yes | Yes | Yes |

### 9.4 Requirements and the trapdoor one-way function

Diffie and Hellman set out the requirements for a public-key cryptosystem:

1. It is computationally **easy** for a party B to generate a key pair ($PU_b$, $PR_b$).
2. It is easy for a sender A, knowing $PU_b$ and M, to generate the ciphertext $C = E(PU_b, M)$.
3. It is easy for B to decrypt: $M = D(PR_b, C)$.
4. It is **computationally infeasible** for an opponent, knowing $PU_b$, to determine $PR_b$.
5. It is computationally infeasible for an opponent, knowing $PU_b$ and C, to recover M.
6. (Useful) Either key can be used for encryption with the other used for decryption: $M = D(PR_b, E(PU_b, M)) = D(PU_b, E(PR_b, M))$. This is needed for signatures.

These requirements boil down to finding a **trapdoor one-way function**: a function f where $Y = f(X)$ is easy to compute for every X, but $X = f^{-1}(Y)$ is computationally infeasible for almost all Y **unless** some secret extra information (the "trapdoor") is known, in which case the inverse is easy. For RSA the one-way direction is multiplying two primes (and modular exponentiation) and the trapdoor is knowledge of the prime factors.

### 9.5 Cryptanalysis and misconceptions

- **Brute-force attack** on the private key is defeated by using large keys, but large keys slow down encryption, so key size is a trade-off.
- **Computing the private key from the public key** – not proven infeasible for any algorithm, but believed to be (for example factoring for RSA).
- **Probable-message attack** – if the message is short or predictable (such as a 56-bit key), the attacker encrypts all likely messages with the public key and compares them to the ciphertext. Defence: **pad** short messages with random bits.
- **Misconception:** public-key encryption is "more secure" than symmetric encryption. No – security depends on key length and computational work.
- **Misconception:** public-key encryption makes symmetric encryption obsolete. No – its computational cost keeps symmetric ciphers essential.
- **Misconception:** key distribution is trivial with public keys. No – you must be sure that a public key really belongs to its claimed owner. A **public-key certificate** signed by a trusted certificate authority provides that assurance.

## 9.6 Mathematical background used by the algorithms

| Concept | Statement |
|---|---|
| Modular arithmetic | a mod n is the remainder when a is divided by n. $a \equiv b \pmod n$ means n divides (a − b). $(a \cdot b) \bmod n = [(a \bmod n)(b \bmod n)] \bmod n$. |
| Prime and gcd | A prime p has only the factors 1 and p. gcd(a, b) is the largest integer dividing both; a and b are **relatively prime** if gcd(a, b) = 1. |
| Multiplicative inverse | If gcd(a, n) = 1 there is a unique $a^{-1}$ with $a \cdot a^{-1} \equiv 1 \pmod n$, found with the **extended Euclidean algorithm**. |
| Euler's totient ϕ(n) | Number of positive integers less than n that are relatively prime to n. ϕ(p) = p − 1 for prime p; ϕ(pq) = (p − 1)(q − 1) for distinct primes p, q. |
| Fermat's theorem | If p is prime and gcd(a, p) = 1, then $a^{p-1} \equiv 1 \pmod p$. |
| Euler's theorem | For every a and n that are relatively prime, $a^{\phi(n)} \equiv 1 \pmod n$. |
| Primitive root | a is a primitive root of prime p if the powers a, $a^2$, …, $a^{p-1} \pmod p$ are all different, i.e. generate every value 1 … p−1. |
| Discrete logarithm | Given a primitive root a of p and a value b, the unique i with $b \equiv a^i \pmod p$ is the discrete log of b. Computing i is believed to be hard for large p; computing $a^i$ mod p is easy. |

### Fast modular exponentiation (square-and-multiply)

Computing $a^b$ mod n by repeated multiplication is infeasible for large b. Instead, write b in binary and use repeated squaring, reducing mod n at each step, so that only about $\log_2 b$ squarings and multiplications are needed:

```
def modexp(a, b, n):        # returns a^b mod n
    result = 1
    a = a % n
    while b > 0:
        if b & 1:           # current bit of b is 1
            result = (result * a) % n
        a = (a * a) % n     # square
        b >>= 1
    return result
```

# 10. RSA Algorithm

## 10.1 Overview

RSA was published in 1977 by **Ron Rivest, Adi Shamir and Leonard Adleman** at MIT and is the most widely used public-key scheme. It is a **block cipher in which the plaintext and ciphertext are integers between 0 and n − 1** for some n; a block of plaintext must therefore be numerically less than n (in practice the block size is a little under the bit length of n). Its security rests on the difficulty of **factoring** the product of two large primes.

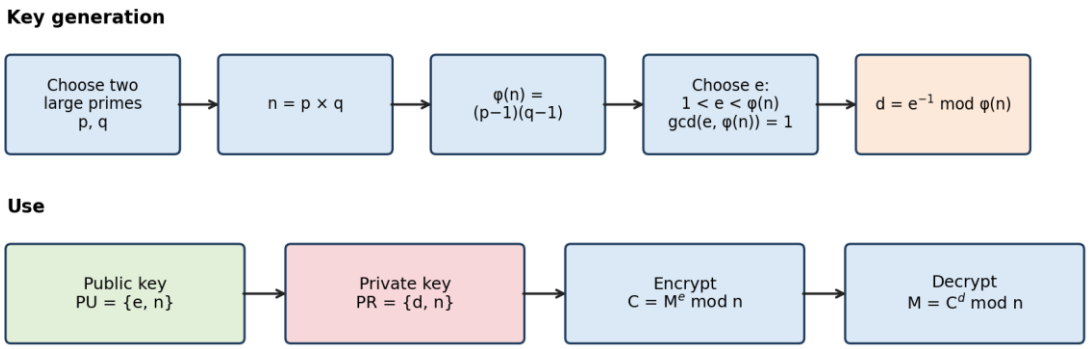



**Figure 14:** *RSA key generation, encryption and decryption.*

## 10.2 The algorithm

| Step | Description |
|---|---|
| **Key generation (by user A)** | |
| 1 | Select two large **primes** p and q, p ≠ q (kept secret). |
| 2 | Calculate **n = p × q**. |
| 3 | Calculate **ϕ(n) = (p − 1)(q − 1)**. |
| 4 | Select an integer **e** such that **gcd(ϕ(n), e) = 1** and 1 < e < ϕ(n). |
| 5 | Calculate **d** such that **d · e ≡ 1 (mod ϕ(n))**, i.e. d = $e^{-1}$ mod ϕ(n). |
| 6 | **Public key PU = {e, n}**; **private key PR = {d, n}**. (p, q and ϕ(n) are discarded or kept very secret.) |
| **Encryption** | Plaintext M < n. Ciphertext **C = $M^e$ mod n** |
| **Decryption** | Ciphertext C. Plaintext **M = $C^d$ mod n** |

## 10.3 Why decryption works

We need $(M^e)^d$ mod n = M. Since ed ≡ 1 (mod ϕ(n)), we can write ed = 1 + k·ϕ(n) for some integer k. For M relatively prime to n, Euler's theorem gives $M^{\phi(n)} \equiv 1 \pmod n$, so

$$M^{ed} = M^{1 + k\varphi(n)} = M \cdot (M^{\varphi(n)})^k \equiv M \cdot 1^k = M \pmod n$$

The result also holds when M shares a factor with n (use Fermat's theorem separately mod p and mod q and the Chinese Remainder Theorem), so it holds for all $0 \le M < n$. The roles of e and d are symmetric, so the same algebra shows that signing with d and verifying with e works too.

## 10.4 Worked example

> **Example (Stallings-style small numbers)**
>
> 1. Select primes **p = 17** and **q = 11**.
> 2. $n = p \times q = 17 \times 11 = \mathbf{187}$.
> 3. $\phi(n) = (17 - 1)(11 - 1) = 16 \times 10 = \mathbf{160}$.
> 4. Choose **e = 7**: gcd(7, 160) = 1 and $1 < 7 < 160$.
> 5. Find d with $d \cdot 7 \equiv 1 \pmod{160}$. Since $23 \times 7 = 161 = 160 + 1$, **d = 23**.
> 6. Public key PU = {7, 187}; private key PR = {23, 187}.

**Encrypt M = 88** ($88 < 187$): $C = 88^7 \bmod 187$. Use repeated squaring:

```
88^1 mod 187 = 88
88^2 mod 187 = 7744 mod 187 = 77
88^4 mod 187 = 77^2  = 5929 mod 187 = 132
88^7 = 88^4 * 88^2 * 88^1  (since 7 = 4 + 2 + 1)
     = (132 * 77) * 88 mod 187
     132 * 77 = 10164 ;  10164 mod 187 = 66
     66 * 88  = 5808  ;  5808  mod 187 = 11
Ciphertext C = 11
```

**Decrypt C = 11:** $M = 11^{23} \bmod 187$. Since 23 = 16 + 4 + 2 + 1:

```
11^1  mod 187 = 11
11^2  mod 187 = 121
11^4  mod 187 = 121^2 = 14641 mod 187 = 55
11^8  mod 187 = 55^2  = 3025  mod 187 = 33
11^16 mod 187 = 33^2  = 1089  mod 187 = 154
11^23 = 11^16 * 11^4 * 11^2 * 11^1
      = 154 * 55 * 121 * 11 mod 187 = 88
Plaintext M = 88  (recovered)
```

**Digital signature with the same keys:** $S = M^d \bmod n = 88^{23} \bmod 187$ (computed with the private key); anyone verifies by checking $S^e \bmod n = 88$.

## 10.5 Computational aspects

- **Encryption/decryption:** use square-and-multiply. Choosing a small **public exponent** such as e = 3, 17 or **65537 ($2^{16}$+1)**, which has only two 1-bits, makes encryption/verification fast. The private exponent d is large, so decryption is slower; it is speeded up about four times using the **Chinese Remainder Theorem** with p and q.
- **Key generation:** pick random odd numbers of the right size and test them for primality with a probabilistic test such as **Miller–Rabin** until two primes are found. Then compute d with the extended Euclidean algorithm.
- p and q should be of similar size (each about half the bits of n), and both $p - 1$ and $q - 1$ should contain a large prime factor.

## 10.6 Security of RSA

- **Brute force:** try all possible private keys; defeated by large d (large n).
- **Mathematical attacks:** equivalent to factoring n = pq: once p and q are known, $\phi(n)$ and then d follow. The best factoring methods (the general number field sieve) are sub-exponential but still impractical for large n. RSA-768 (768 bits) was factored in 2009 after years of effort; **at least 2048-bit moduli** are recommended today (1024-bit is considered unsafe).
- **Timing attacks:** Kocher (1995) showed that the time taken by decryption can leak d. Countermeasures: constant-time exponentiation, random delays, and **blinding** (multiply the ciphertext by a random value before exponentiating).
- **Chosen-ciphertext attacks:** plain ("textbook") RSA is malleable: from $C = M^e$ one can build $C \cdot 2^e$ which decrypts to 2M. The defence is **padding schemes** such as **OAEP** (encryption) and **PSS** (signatures), which should always be used in practice.

# 11. ElGamal Cryptography

Taher ElGamal (1984) devised a public-key scheme based on the **discrete logarithm problem**. It uses the same mathematics as Diffie–Hellman and underlies the ElGamal signature scheme and the Digital Signature Standard (DSS), and it is used in PGP/GnuPG. Here the sender uses a fresh random number for every message so that the same plaintext gives a different ciphertext each time (a **probabilistic** cipher).

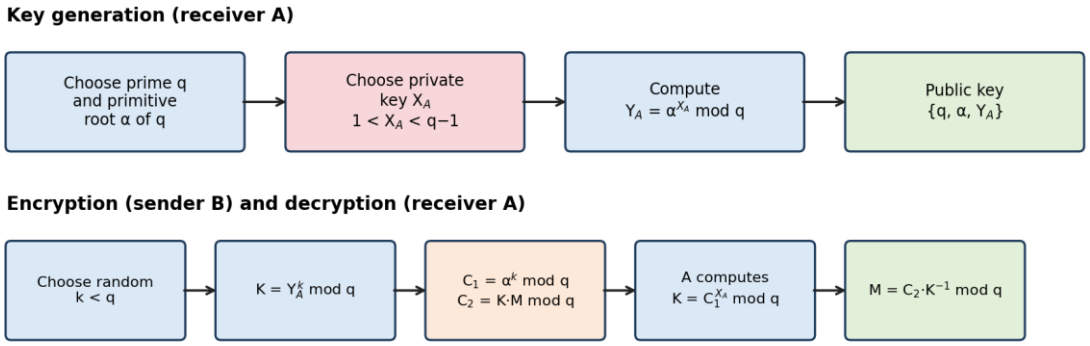



**Figure 15:** *ElGamal key generation, encryption and decryption.*

## 11.1 The algorithm

| Phase | Steps |
|---|---|
| **Global elements** | A large prime q and a primitive root α of q (α < q). |
| **Key generation by receiver A** | 1. Select a random private key $\mathbf{X_A}$ with $1 < X_A < q - 1$.<br>2. Compute $\mathbf{Y_A = \alpha^{X_A} \bmod q}$.<br>3. **Public key** = {q, α, $Y_A$}; **private key** = $X_A$. |
| **Encryption by sender B** (message M < q) | 1. Choose a random integer **k**, $1 \le k \le q - 1$.<br>2. Compute the one-time key $\mathbf{K = (Y_A)^k \bmod q}$.<br>3. Ciphertext is the pair $\mathbf{(C_1, C_2)}$ where $\mathbf{C_1 = \alpha^k \bmod q}$ and $\mathbf{C_2 = K \cdot M \bmod q}$. |
| **Decryption by A** | 1. Recover the key: $\mathbf{K = (C_1)^{X_A} \bmod q}$.<br>2. Compute $\mathbf{M = (C_2 \cdot K^{-1}) \bmod q}$. |

## 11.2 Why it works

$K = (Y_A)^k = (\alpha^{X_A})^k = \alpha^{X_A k}$ and $(C_1)^{X_A} = (\alpha^k)^{X_A} = \alpha^{k X_A}$. The sender (who knows k) and the receiver (who knows $X_A$) compute the **same K**, which is the same idea as Diffie–Hellman. K acts as a one-time **mask** (multiplicative one-time pad) on M. An attacker sees q, α, $Y_A$, $C_1$, $C_2$, and would need to solve the discrete logarithm of $Y_A$ or $C_1$ to recover K.

## 11.3 Worked example

**Example: q = 19, α = 10**

**Key generation (Alice):** choose $X_A = 5$. $Y_A = 10^5 \bmod 19$. $10^2 = 100 \bmod 19 = 5$; $10^4 = 25 \bmod 19 = 6$; $10^5 = 6 \times 10 = 60 \bmod 19 = \mathbf{3}$.

Public key = {19, 10, 3}; private key = 5.

**Encryption (Bob) of M = 17:** choose k = 6. $K = 3^6 \bmod 19 = 729 \bmod 19 = \mathbf{7}$.

$C_1 = 10^6 \bmod 19 = 3 \times 10 = 30 \bmod 19 = \mathbf{11}$. $C_2 = K \cdot M \bmod 19 = 7 \times 17 = 119 \bmod 19 = \mathbf{5}$.

Ciphertext = (11, 5).

**Decryption (Alice):** $K = 11^5 \bmod 19$: $11^2 = 121 \bmod 19 = 7$; $11^4 = 49 \bmod 19 = 11$; $11^5 = 11 \times 11 = 121 \bmod 19 = \mathbf{7}$.

$K^{-1} \bmod 19$: $7 \times 11 = 77 = 4 \cdot 19 + 1$, so $K^{-1} = 11$. $M = 5 \times 11 = 55 \bmod 19 = \mathbf{17}$. ✓

# 12. Diffie–Hellman Key Exchange

Published in 1976 by Diffie and Hellman, this was the **first public-key algorithm**. It does not encrypt messages; its purpose is to let two users **agree on a shared secret key** over an insecure channel, which can then be used for symmetric encryption. Its security rests on the difficulty of computing **discrete logarithms**. It is used in many protocols (TLS, IPsec/IKE, SSH, Signal).

## 12.1 The algorithm

Both users know two public global values: a prime q and a primitive root α of q (α < q).

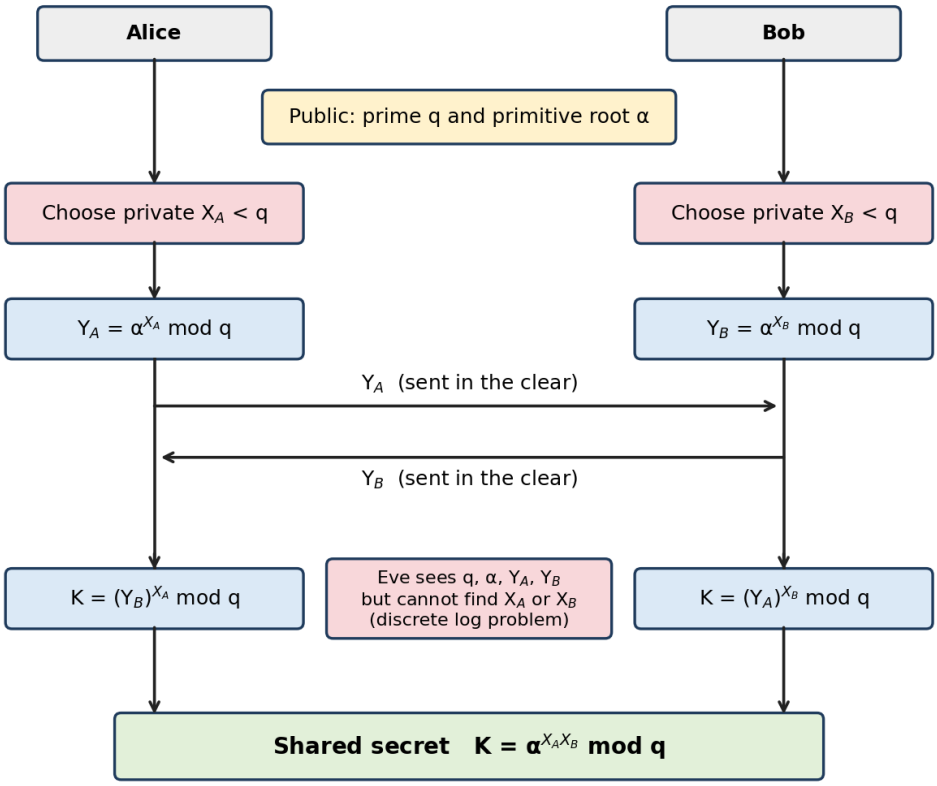



**Figure 16:** *Diffie–Hellman key exchange.*

| Step | User A (Alice) | User B (Bob) |
|---|---|---|
| 1. Private key | Select $X_A < q$ | Select $X_B < q$ |
| 2. Public value | $Y_A = \alpha^{X_A} \bmod q$ | $Y_B = \alpha^{X_B} \bmod q$ |
| 3. Exchange | Send $Y_A$ to Bob | Send $Y_B$ to Alice |
| 4. Secret key | $K = (Y_B)^{X_A} \bmod q$ | $K = (Y_A)^{X_B} \bmod q$ |

Both calculations give the same value because

$$K = (Y_B)^{X_A} \bmod q = (\alpha^{X_B})^{X_A} \bmod q = \alpha^{X_B X_A} \bmod q = (\alpha^{X_A})^{X_B} \bmod q = (Y_A)^{X_B} \bmod q$$

An eavesdropper knows q, α, $Y_A$ and $Y_B$, but to compute K must find $X_A$ or $X_B$, i.e. solve a discrete logarithm. For large q (2048 bits or more) this is infeasible.

## 12.2 Worked example

> **Example: q = 23, α = 5 (5 is a primitive root of 23)**
>
> Alice picks $X_A = 6$ and computes $Y_A = 5^6 \bmod 23$. $5^2 = 25 \bmod 23 = 2$; $5^4 = 4$; $5^6 = 4 \times 2 = \mathbf{8}$.
>
> Bob picks $X_B = 15$ and computes $Y_B = 5^{15} \bmod 23$. $5^8 = 16$; $5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5 = 16 \times 4 \times 2 \times 5 = 640 \bmod 23 = \mathbf{19}$.
>
> They exchange $Y_A = 8$ and $Y_B = 19$ in public.
>
> Alice: $K = 19^6 \bmod 23$. $19^2 = 361 \bmod 23 = 16$; $19^4 = 256 \bmod 23 = 3$; $19^6 = 3 \times 16 = 48 \bmod 23 = \mathbf{2}$.
>
> Bob: $K = 8^{15} \bmod 23$. Since $8 = 2^3$, $8^{15} = 2^{45}$; $2^{11} = 2048 = 89 \cdot 23 + 1 \equiv 1$, so $2^{45} = 2^{44} \cdot 2 \equiv \mathbf{2}$. ✓
>
> Shared secret key K = 2 (a toy value; real keys are hundreds of digits).

## 12.3 Man-in-the-middle attack

Plain Diffie–Hellman is **not authenticated**, so it is vulnerable to an active **man-in-the-middle** attack by an opponent, Darth, who can intercept and replace messages:

1. Darth generates two private values $X_{D1}$ and $X_{D2}$ and the corresponding public values $Y_{D1}$ and $Y_{D2}$.
2. Alice sends $Y_A$ to Bob. Darth intercepts it, and sends $Y_{D1}$ to Bob in its place; Darth computes $K_2 = (Y_A)^{X_D2} \bmod q$.
3. Bob receives $Y_{D1}$ and computes $K_1 = (Y_{D1})^{X_B} \bmod q$. Bob sends $Y_B$ to Alice; Darth intercepts it and sends $Y_{D2}$ to Alice; Darth computes $K_1 = (Y_B)^{X_D1} \bmod q$.
4. Alice computes $K_2 = (Y_{D2})^{X_A} \bmod q$.

Now Bob and Darth share $K_1$, and Alice and Darth share $K_2$. Darth can decrypt each message from Alice, read or change it, and re-encrypt it for Bob, and neither victim notices. **Countermeasure:** authenticate the exchanged values, for example by **digital signatures** or **certificates** on $Y_A$ and $Y_B$ (as in the Station-to-Station protocol and TLS), or by using pre-shared secrets.

# 13. Knapsack Algorithm (Merkle–Hellman)

## 13.1 The knapsack (subset-sum) problem

Given a set of positive integers **A = {$a_1$, $a_2$, …, $a_n$}** and a target sum **S**, the (subset-sum) knapsack problem asks for a subset of A whose elements add up to S. Equivalently: find bits $x_i \in \{0, 1\}$ with

$$S = x_1a_1 + x_2a_2 + \ldots + x_na_n$$

The general problem is **NP-complete**: no efficient algorithm is known, so it looks suitable as a one-way function. The **Merkle–Hellman knapsack cryptosystem** (1978) was the first public-key encryption scheme proposed after Diffie–Hellman. The plaintext bit string is the vector x and the ciphertext is the sum S.

## 13.2 Easy knapsacks: superincreasing sequences

A sequence $a'_1, \ldots, a'_n$ is **superincreasing** if each element is larger than the sum of all previous ones: $a'_i > a'_1 + \ldots + a'_{i-1}$. Such a knapsack is easy: scan from the largest element downwards; if $a'_i \le$ remaining sum, take it ($x_i = 1$) and subtract it, otherwise $x_i = 0$. The solution, if one exists, is unique. This easy knapsack is the **trapdoor**; the public key is a disguised hard-looking version of it.

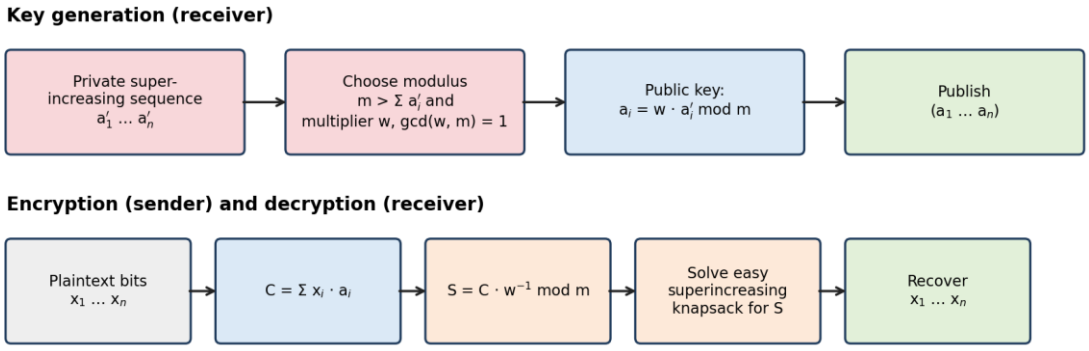



**Figure 17:** *Merkle–Hellman knapsack: key generation, encryption and decryption.*

## 13.3 The algorithm

| Phase | Steps |
|---|---|
| **Key generation** | 1. Choose a **superincreasing** sequence $a' = (a'_1, \ldots, a'_n)$ (the private key).<br>2. Choose a modulus $\mathbf{m > \Sigma a'_i}$.<br>3. Choose a multiplier **w** with **gcd(w, m) = 1**.<br>4. Compute the public sequence $\mathbf{a_i = w \cdot a'_i \bmod m}$.<br>5. **Public key** = $(a_1, \ldots, a_n)$; **private key** = (a′, m, w). |
| **Encryption** | Split the plaintext into n-bit blocks $x = (x_1, \ldots, x_n)$. Ciphertext $\mathbf{C = x_1a_1 + \ldots + x_na_n}$. |
| **Decryption** | 1. Compute $w^{-1} \bmod m$.<br>2. Compute $\mathbf{S = C \cdot w^{-1} \bmod m}$. (Then $S = \Sigma x_i a'_i$.)<br>3. Solve the **easy superincreasing knapsack** (a′, S) to get x. |

Why step 2 works: $C \cdot w^{-1} = \Sigma\, x_i \cdot w\, a'_i \cdot w^{-1} \equiv \Sigma\, x_i a'_i \pmod m$, and because $m > \Sigma\, a'_i$ the true sum is less than m, so the residue equals the sum exactly.

## 13.4 Worked example

**Example: n = 6**

**Private key:** a′ = (1, 2, 4, 10, 20, 40). Superincreasing check: 2 > 1; 4 > 3; 10 > 7; 20 > 17; 40 > 37. ✓

Sum = 77, so choose **m = 110** (> 77). Choose **w = 31** (gcd(31, 110) = 1).

**Public key:** $a_i = 31 \cdot a'_i \bmod 110$:

31·1 = 31; 31·2 = 62; 31·4 = 124 mod 110 = 14; 31·10 = 310 mod 110 = 90; 31·20 = 620 mod 110 = 70; 31·40 = 1240 mod 110 = 30.

Public key a = (31, 62, 14, 90, 70, 30).

**Encrypt** plaintext x = 101110: C = 1·31 + 0·62 + 1·14 + 1·90 + 1·70 + 0·30 = 31 + 14 + 90 + 70 = **205**.

**Decrypt:** $w^{-1} \bmod 110$: 31 × 71 = 2201 = 20·110 + 1, so $w^{-1} = 71$. S = 205 × 71 mod 110 = 14555 mod 110 = **35**.

Solve with a′ = (1, 2, 4, 10, 20, 40) and S = 35: 40 > 35 → $x_6 = 0$; 20 ≤ 35 → $x_5 = 1$, remainder 15; 10 ≤ 15 → $x_4 = 1$, remainder 5; 4 ≤ 5 → $x_3 = 1$, remainder 1; 2 > 1 → $x_2 = 0$; 1 ≤ 1 → $x_1 = 1$, remainder 0.

Plaintext x = (1, 0, 1, 1, 1, 0) = **101110**. ✓

# 14. Comparison, Formula Sheet and Review Questions

## 14.1 Asymmetric algorithms compared

| Algorithm | Hard problem | Public key | Private key | Main use |
|---|---|---|---|---|
| RSA | Integer factorisation | {e, n} | {d, n} | Encryption, signatures, key transport |
| ElGamal | Discrete logarithm | $\{q, \alpha, Y_A\}$ | $X_A$ | Encryption, signatures |
| Diffie–Hellman | Discrete logarithm | $Y_A$, $Y_B$ | $X_A$, $X_B$ | Key exchange only |
| Knapsack (M–H) | Subset-sum (disguised easy knapsack) | $a_1 \dots a_n$ | (a′, m, w) | Historical; broken |

## 14.2 Formula sheet

| Topic | Formula |
|---|---|
| Feistel round | $L_i = R_{i-1}$; $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$ |
| DES round function | $F(R, K) = P(\ S(\ E(R) \oplus K\ )\ )$ (E: 32→48, 8 S-boxes: 48→32, P: 32→32) |
| Triple DES (EDE) | $C = E(K_3, D(K_2, E(K_1, P)))$ |
| AES rounds | $N_r = 10, 12, 14$ for 128, 192, 256-bit keys; round = SubBytes, ShiftRows, MixColumns, AddRoundKey |
| AES field | $GF(2^8)$ with $m(x) = x^8 + x^4 + x^3 + x + 1$ |
| Blowfish F | $F(x) = ((S_1[a] + S_2[b]) \oplus S_3[c]) + S_4[d] \pmod{2^{32}}$ |
| RC5 round | $A = ((A \oplus B) \lll B) + S[2i]$; $B = ((B \oplus A) \lll A) + S[2i+1]$ |
| RSA | $n = pq$; $\phi(n) = (p-1)(q-1)$; $ed \equiv 1 \pmod{\phi(n)}$; $C = M^e \bmod n$; $M = C^d \bmod n$ |
| ElGamal | $Y_A = \alpha^{X_A} \bmod q$; $K = Y_A^k$; $C_1 = \alpha^k$; $C_2 = KM$; $M = C_2 K^{-1} \bmod q$ |
| Diffie–Hellman | $Y_A = \alpha^{X_A}$; $Y_B = \alpha^{X_B}$; $K = Y_B^{X_A} = Y_A^{X_B} = \alpha^{X_A X_B} \bmod q$ |
| Knapsack | $a_i = w\, a'_i \bmod m$; $C = \Sigma\, x_i a_i$; $S = C\, w^{-1} \bmod m$ |